Smart flight individual project



Course title: Intelligent Devices
Date: 19-06-2026
Student: Gijs Brugmans

Table of contents

1. Revision history 3
2. Introduction 4
3. Requirements 5
   - 3.1 Functional- and non-functional requirements 5
   - 3.2 Use-cases 6
4. Analysis 7
   - 4.1 Context diagram 7
   - 4.2 Task decomposition 7
   - 4.3 Task flow 8
5. Design 9
   - 5.1 Class diagram 9
6. Implementation 11
   - 6.1 Drone 11
   - 6.2 IMotor 14
   - 6.3 fakeMotor 15
   - 6.4 IGps 16
   - 6.5 fakeGps 17
   - 6.6 DroneControllerProxy 20
   - 6.7 IReceiver 22
   - 6.8 fakeReceiver 23
   - 6.9 mockController 24
7. Verification 26

1. Revision history

Date	Revision
09-02-2026	Added an introduction
10-02-2026	Written down the functional- and non-functional requirements of the smart flight project.
12-02-2026	Added a context diagram, task decomposition and task flow
24-02-2026	Added a first version of the class diagram
02-03-2026	Remade the context diagram after obtaining feedback on it
04-03-2026	Remade the class diagram while trying to implement SOLID into it
11-06-2026	Remade the functional requirements
12-06-2026	Added the MoSCoW rating to those function requirements
14-06-2026	Added a new and improved class diagram
15-06-2026	Started with giving an explanation about how the system was implemented of all the bits of code that are used in the system
16-06-2026	Finished writing the explanations of all the code and the explanations of the tests
19-06-2026	Added some small feedback points obtained by Lake after a small talk with him, most notably giving more explanation about the class diagram choices and a bigger introduction

2. Introduction

In this report the different steps that will be taken during this individual project are documented. First an analysis will be done of the smart flight project to see what the different parts of the project will contain.

After that the next step will be to design the project with the help of different diagrams. After that the project will be realized and developed with the C++ programming language.

The project that will be discussed, designed and implemented is the development and simulation of a smart flight system. This system should be able to fly by being controlled by a human user, with fake simulated motors, gps system and communication line to know what input the human has made and move and power the motors on this smart flight system accordingly to those inputs.

3. Requirements

3.1 Functional- and non-functional requirements

In this part of the report the requirements for the smart flight system will be written down. These requirements will be divided into functional and non-functional requirements.

Functional requirements will describe how the system should be able to run, while non-functional requirements are qualities like safety features, maintanability and tests.

The functional requirements:

ID	Requirement	MoSCoW
FR01	The system shall be able to move forwards and backwards	Must
FR02	The system shall be able to move sideways to the left and right	Must
FR03	The system shall be able to move up and down	Must
FR04	The system shall contain a flight mode where the drone will be controlled manually by the user and where the user has full control over what the drone does	Must
FR05	The system shall know the location of the drone with the help of a GPS	Should
FR06	The system shall be able to display the location, height and outputed power of the motors in a terminal	Should
FR07	The system shall contain a minimum flying height that the user can't go under to prevent crashing into the ground	Should
FR08	The system shall contain a test flight where it tests if all of the instruments stil work correctly	Could
FR09	The system shall contain a flight mode where the drone will be at a fixed height, but the user will still have manual control over the horizontal movement	Could
FR10	The system shall contain a flight state where the drone stabalises itself to a certain height and have no movement anymore.	Could
FR11	The system shall contain a way to swap between the different flight states while being in the air	Could
FR12	The system shall be able to turn around to the left and right	Would
FR13	The system shall know what angle the drone is facing in degrees going from 0 to 359 with the help of a barometer an a magnetometer.	Would

The non-functional requirements:

ID	Requirement
NFR01	The system shall have a minimal flight height when it is flying and above a certain horizontal speed.
NFR02	The system shall have a quick response time when the user is using the manual controls
NFR03	The accelerometer and GPS shall be able to communicate with each other to be able to calculate the location and speed
NFR04	The system shall be able to swap between the different flight modes quickly.

3.2 Use-cases

In this part of the report the different cases users can run into while using the smart drone can be explained. This has mostly to do with safety or communication. Some of these will be written out in this next part.

Use case 1:
“The system detects that the user wants to activate the system and fly. The system gives the motors the command to start up and go to the minimum flying height and the display shows the basic start up data where the drone is floating on the minimum flying height.”

Use case 2:
“The system detects that the user want the drone to go higher up. The system gives the command to give the motors more power

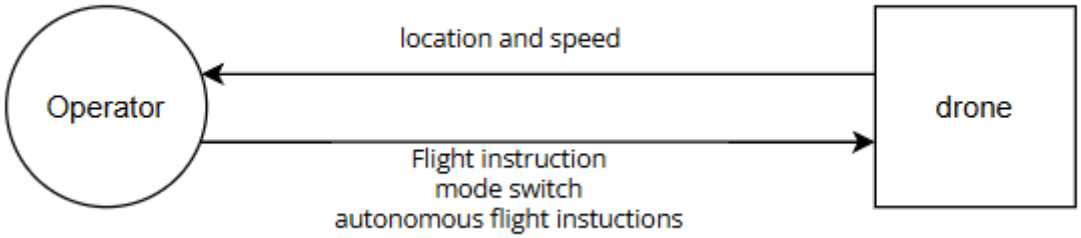
4. Analysis

4.1 Context diagram

The first thing that was done in the analysis is visualizing the system with all the different sub-systems into a context diagram. The different subsystems in the smart flight are:

- Operator: The user giving different instructions to the drone while getting the speed and location of the drone.

With this in mind the context diagram was made that is shown here below.



Picture 1: In the picture above the context diagram of the smart drone is shown

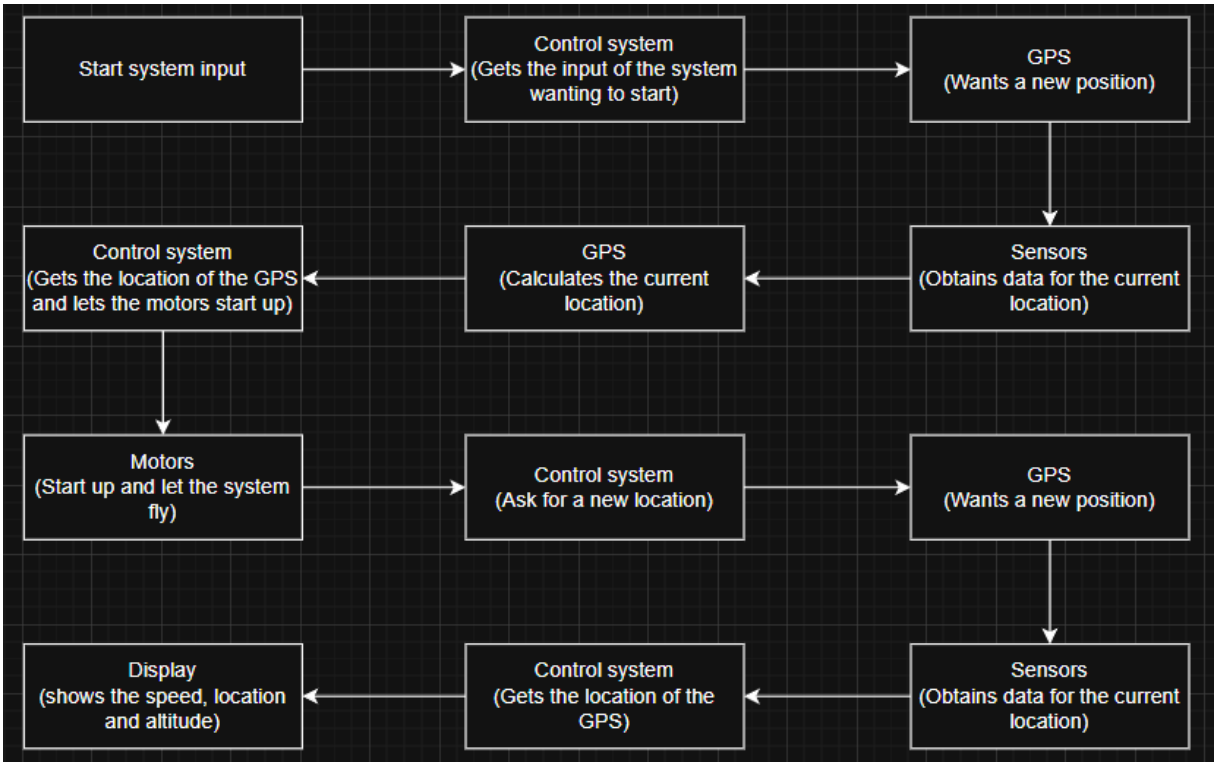
4.2 Task decomposition

In this part of the report all of the different components get divided into different subsystems.

Subsystem	**Description**
Display	Visualizes all the important information of the flight system to the user.
Flight mode struct	Saves the different modes the drone can fly in and makes sure only one is active at a time
Motor	Gives power to move the flight system around in the air.
GPS	Using all the obtained data from the sensors to update the location
DroneControllerProxy	This receives the data coming from the controller controlling the smart flight
Controller	This lets the user control the drone when it is activated and in the air

4.3 Task flow

The task flow visualizes the flow of the different tasks for the system to work for different use cases. The task flow that will be shown will be based on



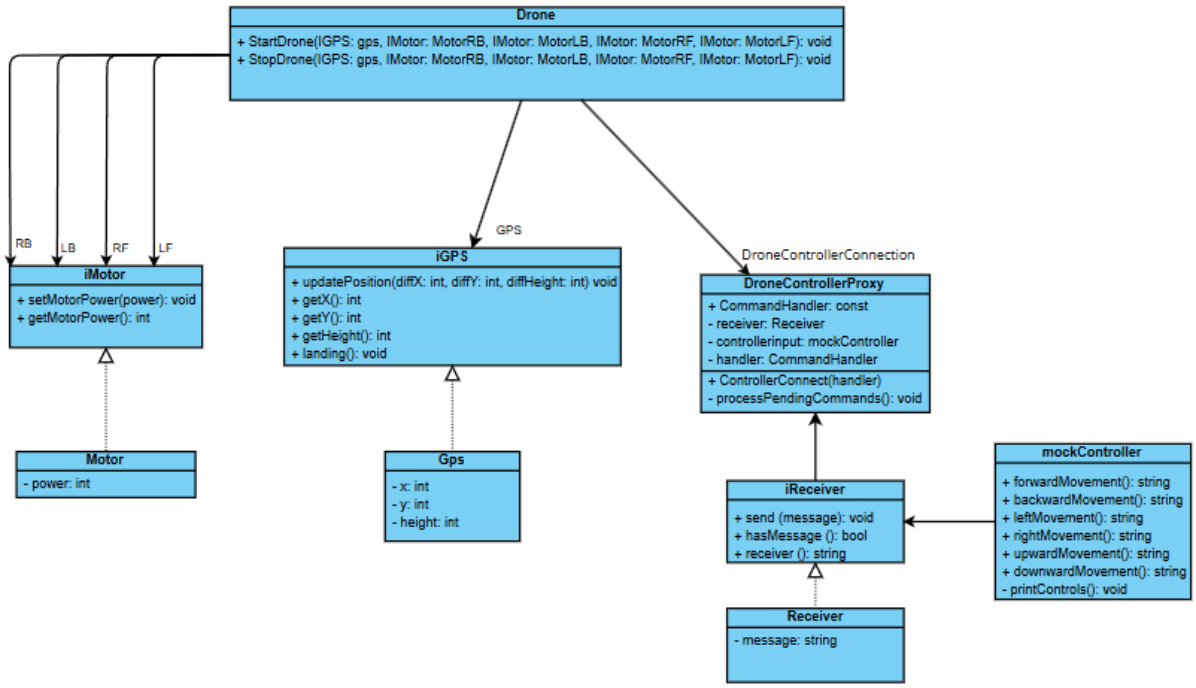
Picture 2: In the picture above the task flow for the use case of starting the system and getting it to a minimum flying height is shown.

5. Design

In this part of the report the design will be decided on. This will show how the system should look when it is created.

5.1 Class diagram

The class diagram shows the different classes of the system and the which classes give or receive data from other classes. The class diagram of the smart flight system is shown below.



Picture 3: In the picture above the class diagram for the smart flight system is shown

As is visible in the picture above there are a loads of different classes in this system, all with their own tasks that they have to execute. This is done because of SOLID, specifically Single responsibility. This is a part of SOLID that says that every class should only be responsible for 1 thing. So the Motor class is only responsible for the motors, and the Receiver class is only responsible for receiving the messages of the mockController.

Another part of SOLID is the Interface Segragation. This is where interfaces have to be as small as possible to make sure that if an interface isn't needed, it can be excluded from the program. This is best showcased with the IMotor and IGps. These could've been combined into one interface called movement, but than if the gps or motors weren't needed that class would've been to big. Instead the class was separated into IMotor and IGps to make sure that they both have their separate tasks to execute.

Lastly of SOLID is Dependency ..., which is when bigger classes have to rely on smaller classes to work. This can be fixed by working with abstractions of the class instead of the class itself. This can best be seen with the DroneControllerProxy class where an abstraction of the Receiver and the mockController classes are used.

The table of this class diagram is shown below. Here the responsibilities of the classes and the collaborations of the different classes is explained some more.

Class	**Responsibility**	**Collaborates with**
Drone	Making sure the power of the motors gets updated and the location gets updated with help of the gps based on the commands it gets from the DroneControllerProxy	Motors, GPS, DroneControllerProxy
Gps	Gives the new location of the drone. In this project because of the lack of hardware to use this GPS is a fake one simulated as if it was a real one	Drone
Motor	Gives the power of the motors. In this project because of the lack of hardware to use these Motors are fake that are simulated as if they were real	Drone
DroneControllerProxy	Creates a connection between the drone and receiver of the commands of the controller	Drone, fakeReceiver
Receiver	Receives the moving commands coming from the mock controller. In this project because of the lack of hardware to use this receiver is a fake one simulated as if it was a real one.	DroneControllerProxy
mockController	Decides what moves are executed based on the input from the user	Receiver

6. Implementation

In this part of the report the code will get explained. The purpose of every class will be explained in more detail and also every function will be showed. Every class will first be show the header file where all the functions that happen are shown, after that the actual c++ file will be shown.

6.1 Drone

The drone class is the main class of this system. It is where all the data and instructions arrive or get send from.

```
#ifndef DRONE_H
#define DRONE_H

#include "IGps.h"
#include "IMotor.h"

#define MOVE_FORWARD 1
#define MOVE_BACKWARD -1
#define MOVE_RIGHT 1
#define MOVE_LEFT -1
#define MOVE_UP 1
#define MOVE_DOWN -1

#define STRONG_MOTOR_POWER_FORWARD 50
#define SOFT_MOTOR_POWER_FORWARD 25

#define STRONG_MOTOR_POWER_BACKWARD 50
#define SOFT_MOTOR_POWER_BACKWARD 25

#define STRONG_MOTOR_POWER_LEFT 50
#define SOFT_MOTOR_POWER_LEFT 25

#define STRONG_MOTOR_POWER_RIGHT 50
#define SOFT_MOTOR_POWER_RIGHT 25

#define MOTOR_UP_POWER 50

#define MOTOR_DOWN_POWER 10

#define MOTOR_STARTUP_POWER 40

#define MOTOR_FLOATING_POWER 20

#define MOTOR_LANDING_POWER 5

#define MOTOR_OFF 0

void StartDrone(IGps& gps, IMotor& MotorRB, IMotor& MotorLB, IMotor& MotorRF, IMotor& MotorLF);
void StopDrone(IGps& gps, IMotor& MotorRB, IMotor& MotorLB, IMotor& MotorRF, IMotor& MotorLF);

#endif
```

Picture 4: In this picture above drone.h is shown

In the picture above the magic numbers that are used within the drone.cpp file are shown. These are used for better and easier maintainability if something needs to be changed. This file also create the two functions that start of end the life cycle of the system. This file is mostly here so that the drone.cpp file looks a lot cleaner than it would otherwise.

The defines are here to give the motors predefined powers without using magic numbers.

The pointers that are seen in the 2 functions are a type of smart pointers called unique pointers. These pointers make sure that every part has exclusive ownership of that part. This avoids any collisions if they should happen and it makes it clear which owner can edit what part.

```
void StartDrone(IGps& gps, IMotor& MotorRB, IMotor& MotorLB, IMotor& MotorRF, IMotor& MotorLF)
{
    std::cout << "Drone starting up \n";
    MotorRB.setMotorPower(MOTOR_STARTUP_POWER);
    MotorLB.setMotorPower(MOTOR_STARTUP_POWER);
    MotorRF.setMotorPower(MOTOR_STARTUP_POWER);
    MotorLF.setMotorPower(MOTOR_STARTUP_POWER);
    std::cout   << "Motors - RB: " << MotorRB.getMotorPower()
                << " LB: " << MotorLB.getMotorPower()
                << " RF: " << MotorRF.getMotorPower()
                << " LF: " << MotorLF.getMotorPower() << "\n";
    Sleep(1000);
    MotorRB.setMotorPower(MOTOR_FLOATING_POWER);
    MotorLB.setMotorPower(MOTOR_FLOATING_POWER);
    MotorRF.setMotorPower(MOTOR_FLOATING_POWER);
    MotorLF.setMotorPower(MOTOR_FLOATING_POWER);
    gps.updatePosition(0, 0, 5);
    std::cout << "Drone has reached starting position: x=" << gps.getX()
              << " y=" << gps.getY()
              << " Height=" << gps.getHeight() << "\n";
    std::cout   << "Motors - RB: " << MotorRB.getMotorPower()
                << " LB: " << MotorLB.getMotorPower()
                << " RF: " << MotorRF.getMotorPower()
                << " LF: " << MotorLF.getMotorPower() << "\n";
}
```

Picture 5: In the picture above the worked out StartDrone function is shown

In the picture above the StartDrone function is shown. It is just a bunch of lines that the drone executes when starting up the system before it becomes controllable by the human user.

```
void StopDrone(IGps& gps, IMotor& MotorRB, IMotor& MotorLB, IMotor& MotorRF, IMotor& MotorLF)
{
    std::cout << "Drone starting to land \n";
    MotorRB.setMotorPower(MOTOR_LANDING_POWER);
    MotorLB.setMotorPower(MOTOR_LANDING_POWER);
    MotorRF.setMotorPower(MOTOR_LANDING_POWER);
    MotorLF.setMotorPower(MOTOR_LANDING_POWER);
    std::cout   << "Motors - RB: " << MotorRB.getMotorPower()
                << " LB: " << MotorLB.getMotorPower()
                << " RF: " << MotorRF.getMotorPower()
                << " LF: " << MotorLF.getMotorPower() << "\n";
    Sleep(1000);
    std::cout << "Drone has landed\n";
    MotorRB.setMotorPower(MOTOR_OFF);
    MotorLB.setMotorPower(MOTOR_OFF);
    MotorRF.setMotorPower(MOTOR_OFF);
    MotorLF.setMotorPower(MOTOR_OFF);
    std::cout   << "Motors - RB: Off LB: Off RF: Off LF: Off\n";
    gps.landing();
    std::cout << "Landing position: X=" << gps.getX()
              << " Y=" << gps.getY()
              << " Height=" << gps.getHeight() << "\n";
}
```

Picture 6: In the picture above the worked out StopDrone function is shown.

In the picture above the StopDrone function is shown. Just like the StartDrone function it is just a bunch of lines that the drone executes before it completely shuts down the system.

```cpp
#include <iostream>
#ifdef _WIN32
#include <conio.h>
#include <windows.h>
#endif
#include "DroneControllerProxy.h"
#include "drone.h"
#include <memory>

int main()
{
    auto Gps = createGps();
    auto MotorRB = createMotor();
    auto MotorLB = createMotor();
    auto MotorRF = createMotor();
    auto MotorLF = createMotor();
    StartDrone(*Gps, *MotorRB, *MotorLB, *MotorRF, *MotorLF);
```

Picture 7: In the picture above the starting up part of the system is shown

In the picture above the include of drone.cpp and the starting up phase of the system is shown. When starting up the system it first creates a gps and the 4 motors that are needed for the rest of the system.

The auto that stands for the names is there to let the compiler know it can write the full type of the thing that is being created, which is a unique_ptr (unique pointer).

```cpp
DroneControllerProxy proxy([&](const std::string& cmd)
{
    if (cmd == "FORWARD")
    {
        Gps->updatePosition(MOVE_FORWARD, 0, 0);
        MotorRB->setMotorPower(STRONG_MOTOR_POWER_FORWARD);
        MotorLB->setMotorPower(STRONG_MOTOR_POWER_FORWARD);
        MotorRF->setMotorPower(SOFT_MOTOR_POWER_FORWARD);
        MotorLF->setMotorPower(SOFT_MOTOR_POWER_FORWARD);
    }
    else if (cmd == "BACKWARD")
    {
        Gps->updatePosition(MOVE_BACKWARD, 0, 0);
        MotorRB->setMotorPower(SOFT_MOTOR_POWER_BACKWARD);
        MotorLB->setMotorPower(SOFT_MOTOR_POWER_BACKWARD);
        MotorRF->setMotorPower(STRONG_MOTOR_POWER_BACKWARD);
        MotorLF->setMotorPower(STRONG_MOTOR_POWER_BACKWARD);
    }
    else if (cmd == "LEFT")
    {
        Gps->updatePosition(0, MOVE_LEFT, 0);
        MotorRB->setMotorPower(STRONG_MOTOR_POWER_LEFT);
        MotorLB->setMotorPower(SOFT_MOTOR_POWER_LEFT);
        MotorRF->setMotorPower(STRONG_MOTOR_POWER_LEFT);
        MotorLF->setMotorPower(SOFT_MOTOR_POWER_LEFT);
    }
    else if (cmd == "RIGHT")
    {
        Gps->updatePosition(0, MOVE_RIGHT, 0);
        MotorRB->setMotorPower(SOFT_MOTOR_POWER_RIGHT);
        MotorLB->setMotorPower(STRONG_MOTOR_POWER_RIGHT);
        MotorRF->setMotorPower(SOFT_MOTOR_POWER_RIGHT);
        MotorLF->setMotorPower(STRONG_MOTOR_POWER_RIGHT);
    }
    else if (cmd == "UP")
    {
        Gps->updatePosition(0, 0, MOVE_UP);
        MotorRB->setMotorPower(MOTOR_UP_POWER);
        MotorRF->setMotorPower(MOTOR_UP_POWER);
        MotorLB->setMotorPower(MOTOR_UP_POWER);
        MotorLF->setMotorPower(MOTOR_UP_POWER);
    }
    else if (cmd == "DOWN")
    {
        Gps->updatePosition(0, 0, MOVE_DOWN);
        MotorRB->setMotorPower(MOTOR_DOWN_POWER);
        MotorRF->setMotorPower(MOTOR_DOWN_POWER);
        MotorLB->setMotorPower(MOTOR_DOWN_POWER);
        MotorLF->setMotorPower(MOTOR_DOWN_POWER);
    }
});
```

Picture 8: In the picture above the movement options are shown

In the picture above the manual movement options that the drone has are shown. These movements are coming from the DroneControllerProxy and here in the drone class they trigger the update of the motor power and an update of the coordinates that get maintained by the gps. Every move here is predefined, but the user has to decide which move the drone is going to make

```
while (true)
{
    int key = _getch();
    if (key == 'q' || key == 'Q')
    {
        StopDrone(*Gps, *MotorRB, *MotorLB, *MotorRF, *MotorLF);
        break;
    }

    std::string message = proxy.handleInput(key);
    std::cout << message << "\n";

    std::cout << "Position: x=" << Gps->getX()
              << " y=" << Gps->getY()
              << " Height=" << Gps->getHeight() << "\n";
    std::cout << "Motors - RB: " << MotorRB->getMotorPower()
              << " LB: " << MotorLB->getMotorPower()
              << " RF: " << MotorRF->getMotorPower()
              << " LF: " << MotorLF->getMotorPower() << "\n";
}

return 0;
```

Picture 9: In the picture above the final part of the system is shown.

In the picture above the part of the system is shown where it becomes clear what is done if the system is going to shut down. This shutting down is triggered by the human user pressed q, which can only be triggered if the system is actively running.

If this is the case it first executes the StopDrone function, and after that it send a couple of messages, like the landed coordinates of the drone and after that it shuts down the system and the only way to start it back up again is to do that manually.

6.2 IMotor

The IMotor class is a class that serves as an abstract interface for the creation and behavior of the motors that are getting used by the system.

```
#ifndef IMOTOR_H
#define IMOTOR_H
#include <memory>

class IMotor
{
public:
    virtual ~IMotor() = default;
    virtual void setMotorPower(int power) = 0;
    virtual int getMotorPower() const = 0;
};

std::unique_ptr<IMotor> createMotor();

#endif
```

Picture 10: In the picture above IMotor.h is shown

In the picture above the function that IMotor contains are shown. The setMotorPower function is to set the power of the motor, this can range from 0 power to 100 and can also be predefined based on what movement is done.

The getMotorPower function is used to actually be able to print the power of the motor inside the terminal.

The createMotor function is used to create the 4 motors used by the system. They all contain a unique pointer to give them sole ownership over their specific motor and to not cause any data issues

6.3 fakeMotor

The fakeMotor class is to actually execute the implementation of the IMotor class.

```h
#ifndef FAKEMOTOR_H
#define FAKEMOTOR_H

#include "IMotor.h"

#define MAX_POWER 100
#define MIN_POWER 0

class fakeMotor : public IMotor
{
public:
    fakeMotor();
    void setMotorPower(int power) override;
    int getMotorPower() const override;

private:
    int power;
};
#endif
```

Picture 11: In the picture above fakeMotor.h is shown

In the picture above the fakeMotor header file is shown. It is shown that the fakeMotor as a direct connection with IMotor. Here both of the functions that get called in the interface class are also called, including the variable 'power' that gets used inside the setMotorPower function.

Because the variable 'power' is only used in this class it is set in private and therefore encapsuled to not be used by other classes. This way not other class can interfere with the data around power.

The defines are again here as to make the c file look more cleanly. The purpose of these two defines is to give the maximum and minimum power of the motor.

```cpp
#include "fakeMotor.h"

#include <memory>

fakeMotor::fakeMotor()
    : power(MIN_POWER)
{
}

void fakeMotor::setMotorPower(int power)
{
    if (power < MIN_POWER)
        this->power = MIN_POWER;
    else if (power > MAX_POWER)
        this->power = MAX_POWER;
    else
        this->power = power;
}

int fakeMotor::getMotorPower() const
{
    return power;
}

std::unique_ptr<IMotor> createMotor()
{
    return std::make_unique<fakeMotor>();
}
```

Picture 12: In the picture above fakeMotor.cpp is shown

In the picture above the c file of the fakeMotor class is shown.

The first function set the power of the motor immediately to 0, to make sure that the motors always have a power and that that power is always predefined to 0 as to not make any mistakes with the coordinates

The setMotorPower function is defined next and can be determined by just calling the function. It has a little if else statement because if the power that is given by the system exceeds the maximum or minimum power it needs to be set to the maximum or minimum power that can be given by the motors. If the power is between the lines of the minimum and maximum power the system just executes it like normat.

The getMotorPower function is here to just return the power for it to be printed

The createMotor function returns a unique pointer that can be used by one of the 4 motors in the system to decide what power that specific motor will have at any given time or move that will be made.

6.4 IGps

The IGps class is a class that serves as an abstract interface for the creation and behavior of the gps that gets used by the system.

```
#ifndef IGPS_H
#define IGPS_H

#include <memory>

class IGps
{
public:
    virtual ~IGps() = default;
    virtual void updatePosition(int diffX, int diffY, int diffHeight) = 0;
    virtual int getX() const = 0;
    virtual int getY() const = 0;
    virtual int getHeight() const = 0;
    virtual void landing() = 0;
};

std::unique_ptr<IGps> createGps();

#endif
```

Picture 13: In the picture above IGps.h is shown

In the picture above the function that the Gps interface looks over are shown.

The updatePosition function uses the variable ‘diffX’, ‘diffY’, and ‘diffHeight’ to make sure that the coordinates that get shown in the terminal are always up-to-date with the real-time coordinates.

The getX function gives a way to save the X-coordinate so that it can be updated with the differences made on the x-axile by moving the drone on that axile. These updates can occur if the drone moves up or down.

The getY function gives a way to save the Y-coordinate so that it can be updated with the difference made on the y-axile by moving the drone on that axile. These updates can occur if the drone moves to the left or right.

The getHeight function gives a way to save the height the drone is at so that it can be updated with the difference made on the height by moving the drone on that axile. These updates can occur if the drone moves up or down.

The landing function gives the drone a way to land without falling to the ground because of the minimum flying height that it has. If this function didn’t exist it would shut off at a height of 5 and then just fall to the ground.

The createGps function is used to create the Gps used by the system. The gps contains a unique pointer to give it sole ownership over its own gps and to not cause any data issues.

6.5 fakeGps

The fakeGps class is to actually execute the implementation of the IGps class.

```
#ifndef FAKEGPS_H
#define FAKEGPS_H

#include "IGps.h"

#define MINIMUM_HEIGHT 5

class fakeGps : public IGps
{
public:
    fakeGps();
    void updatePosition(int diffX, int diffY, int diffHeight) override;
    void landing();
    int getX() const override;
    int getY() const override;
    int getHeight() const override;

private:
    int x;
    int y;
    int height;
};

#endif
```

Picture 14: In the picture above fakeGps.h is shown

As is visible in the picture above all of the functions except for the create function are called in this header file, combined with the 3 values for the coordinates of the gps, that being x, y, and height.

These values are made private so that no other class can mess with these values just like how power was set to private inside the fakeMotor class.

The defines are also set inside the header file to make the fakeGps.cpp file look more cleanly and make it easier to edit that file.

```cpp
#include "fakeGps.h"

#include <iostream>

#include <memory>

fakeGps::fakeGps()
    : x(0), y(0), height(0)
{
}

void fakeGps::updatePosition(int diffX, int diffY, int diffHeight)
{
    x += diffX;
    y += diffY;
    height += diffHeight;
    if (height < MINIMUM_HEIGHT)
    {
        height = MINIMUM_HEIGHT;
        std::cout << "Stopped from going lower as you reached the minimum flying height \n";
    }
}
```

Picture 15: In the picture above the start of fakeGps.cpp is shown.

In the picture above the first part of the implementation of the function used by the fakeGps class are shown.

The first function is just here to set the values of the coordinates used by the gps. This way the other functions that might edit these coordinates have a value to edit.

The updatPosition function is also shown here. This function adds or subtracts the movement coordinates from the coordinates that display the placement of the drone.

Here it is also shown that the drone has a built in minimum flying height. If the user tries to go under this height the drone will stop it from doing so and say to the human user that it has reached the drones minimum flying height.

```cpp
int fakeGps::getX() const
{
    return x;
}

int fakeGps::getY() const
{
    return y;
}

int fakeGps::getHeight() const
{
    return height;
}

void fakeGps::landing()
{
    height = 0;
}

std::unique_ptr<IGps> createGps()
{
    return std::make_unique<fakeGps>();
}
```

Picture 16: In the picture above the remaining part of fakeGps.cpp is shown.

In the picture above the remaining part of the fakeGps class functions implementation is shown.

The first three function all do the same thing, just for different coordinates. They all return the coordinate of the placement of the drone so that they can be printed onto the terminal.

The fourth function, landing, gives the drone a way to bypass the minimum flying height when it wants to land when the system is told to shut down. If this function wasn't here it wouldn't be able to bypass the minimum flying height and the drone would shut off and fall down from the set minimum flying height onto the ground.

The createGps function returns a unique pointer that can be used by the gps of the system to return the coordinates from this class.

6.6 DroneControllerProxy

The DroneControllerProxy class' usage is to convert keyboard commands coming from the terminal into actual string commands that can be read out by the rest of the system to be executed.

```
#ifndef DRONECONTROLLERPROXY_H
#define DRONECONTROLLERPROXY_H

#include <functional>
#include <string>
#include "mockController.h"
#include "fakeReceiver.h"

class DroneControllerProxy
{
public:
    using CommandHandler = std::function<void(const std::string&)>;

    explicit DroneControllerProxy(CommandHandler handler);
    std::string handleInput(int key);

private:
    fakeReceiver receiver;
    mockController controllerinput;
    CommandHandler handler;

    void processPendingCommands();
};

#endif
```

Picture 17: In the picture above DroneControllerProxy.h is shown

As shown in the picture above the class has connection with the mockController and the fakeReceiver where it in a real scenario would receiver these commands from.

It creates a CommandHandler, that can receive the movement commands.

The following members are again created privately since no other class has to influence the data from it. These member are receiver that comes from the class fakeReceiver, controllerinput that comes from the class mockController and the CommandHandler that was already defined earlier

The other function that gets created privately is proccesPendingCommands, which is because this function works as a queue. This queue only needs to be accessed by this class, so it is stored privately to prevent its contained data from being altered or corrupted by other classes that have nothing to do with the function

```
#include "DroneControllerProxy.h"

DroneControllerProxy::DroneControllerProxy(CommandHandler handler)
    : receiver(), controllerinput(receiver), handler(std::move(handler))
{
}
```

Picture 18: In the picture above the first part of DroneControllerProxy.cpp is shown

In the picture above the implementation of the first function gets shown. The CommandHandler gets taken by a constructor to be used in the DroneControllerProxy function, where it gets stored so it can be used at a later time. This function just gives values or places where to get values from to the different members of the class

```
std::string DroneControllerProxy::handleInput(int key)
{
    std::string message;
    switch (key)
    {
        case 'w':
        case 'W':
            message = controllerinput.forwardMovement();
            break;
        case 's':
        case 'S':
            message = controllerinput.backwardMovement();
            break;
        case 'a':
        case 'A':
            message = controllerinput.leftMovement();
            break;
        case 'd':
        case 'D':
            message = controllerinput.rightMovement();
            break;
        case 'u':
        case 'U':
            message = controllerinput.upwardMovement();
            break;
        case 'j':
        case 'J':
            message = controllerinput.downwardMovement();
            break;
        default:
            message = "Unknown key. Use w/s/a/d/u/j or q to quit.";
            break;
    }

    if (message != "Unknown key. Use w/s/a/d/u/j or q to quit.")
    {
        processPendingCommands();
    }

    return message;
}
```

Picture 19: In the picture above the next implemented function of DroneControllerProxy.cpp is shown

In the picture above the implementation of the handleInput function is shown. This function is in short a big switch case that gets triggered based on the letter that gets put into the terminal by the user of the system. If the letter or other character that gets put into the terminal isn't one that is mentioned by the switch case, it goes further into the queue and gives a message that the key is not mapped/unknown by the system.

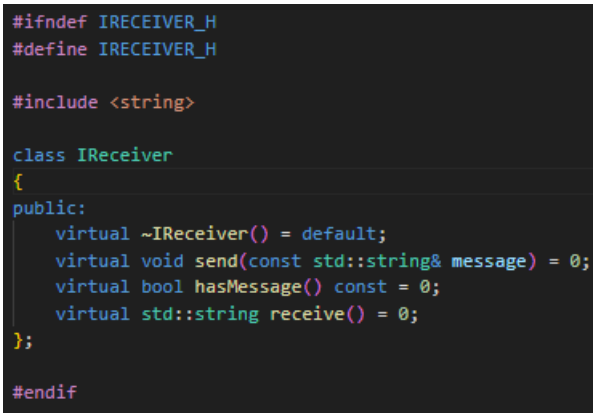
```
void DroneControllerProxy::processPendingCommands()
{
    while (receiver.hasMessage())
    {
        handler(receiver.receive());
    }
}
```

Picture 20: In the picture above the last implemented function of DroneControllerProxy.cpp is shown.

In the picture above the implementation of the processPendingCommands fucntion is shown. This function acts as a queue that can make it so that if the inputs that are made pile up because the system can't process them fast enough, they still get executed when the queue gets to that point.

6.7 IReceiver

The IReceiver class is a class that serves as an abstract interface for the creation and behavior of the fakereceiver that gets used by the system.



```
#ifndef IRECEIVER_H
#define IRECEIVER_H

#include <string>

class IReceiver
{
public:
    virtual ~IReceiver() = default;
    virtual void send(const std::string& message) = 0;
    virtual bool hasMessage() const = 0;
    virtual std::string receive() = 0;
};

#endif
```

Picture 21: In the picture above IReceiver.h is shown.

In the picture above the function that the Gps interface looks over are shown.

The send function is used to regulate the sent messages so that they arrive at the correct end point where they are meant to be.

The hasMessage boolean is used to check if the receiver already has a message it is receiving. If this is not the case it will make sure that receive will be activated so that a new message can be received by the class.

6.8 fakeReceiver

The fakeReceiver class is to actually execute the implementation of the IReceiver class.

```
#ifndef FAKERECEIVER_H
#define FAKERECEIVER_H

#include "IReceiver.h"
#include <queue>
#include <string>

class fakeReceiver : public IReceiver
{
public:
    fakeReceiver() = default;
    void send(const std::string& message) override;
    bool hasMessage() const override;
    std::string receive() override;

private:
    std::queue<std::string> messages;
};

#endif
```

Picture 22: In the picture above fakeReceiver.h is shown

In the picture above the header file of the fakeReceiver class is shown. It contains all the same functions as the IReceiver file and they all stand in public

The member messages was put in private since only the fakeReceiver class has to edit this member and therefore like the other members from other classes it is put to private to make sure that other functions cannot edit or corrupt the data of that member.

```
#include "fakeReceiver.h"
#include <utility>

void fakeReceiver::send(const std::string& message)
{
    messages.push(message);
}

bool fakeReceiver::hasMessage() const
{
    return !messages.empty();
}

std::string fakeReceiver::receive()
{
    if (messages.empty()) return std::string();
    std::string m = messages.front();
    messages.pop();
    return m;
}
```

Picture 23: In the picture above the fakeReceiver.cpp is shown

In the picture above the implementation of the functions that were established inside fakeReceiver.h and the interface are shown.

The send function makes sure that the message gets send, by pushing it onto an internal queue so that the DroneControllerProxy can read it and execute the command that it needs to execute

The hasMessage function check if the receiver has a message to send. Based on the result the boolean either gives back a 0 or a 1. Based on this boolean the receive function will be activated or not.

If the boolean from hasMessage return a 0, the receive function will be activated and wait until it can return a message to be processed and received by the system.

6.9 mockController

The mockController is a fake controller giving a bunch of data to the system that is all predefined. The mockController is the substitute of the controller that should otherwise control the drone. This way the drone can still be simulated without having to edit the physical part of a drone and a controller to the system.

```
#ifndef MOCKCONTROLLER_H
#define MOCKCONTROLLER_H

#include "IReceiver.h"

class mockController
{
public:
    explicit mockController(IReceiver& receiver);
    std::string forwardMovement();
    std::string backwardMovement();
    std::string leftMovement();
    std::string rightMovement();
    std::string upwardMovement();
    std::string downwardMovement();

private:
    IReceiver& receiver;
    void printControls() const;
};

#endif
```

Picture 24: In the picture above mockController.h is shown

In the picture above the header file of the mockController class is shown. It contains all the functions that the mockController has to execute and they all stand in public except for one.

That one function is the printControls function. This one stands in private because nothing needs to change from that function which will be shown and explained a little bit later inside this report.

The other part that is put in private is a unique pointer called receiver which points to the receiver interface. This will make sure that the receiver will print out the controls that are executed.

```
#include "mockController.h"
#include <string>
#include <iostream>
#ifdef _WIN32
#include <conio.h>
#endif

mockController::mockController(IReceiver& receiver)
    : receiver(receiver)
{
    printControls();
}
```

Picture 25: In the picture above the first part of mockController.cpp is shown.

In the picture above the implementation of the mockController function is shown. When the system starts up it will send the printControls function, which is showcased later inside the report, to the receiver to be send through the communication line.

```cpp
std::string mockController::forwardMovement()
{
    receiver.send("FORWARD");
    return "Forward movement";
}

std::string mockController::backwardMovement()
{
    receiver.send("BACKWARD");
    return "Backward movement";
}

std::string mockController::leftMovement()
{
    receiver.send("LEFT");
    return "Left movement";
}

std::string mockController::rightMovement()
{
    receiver.send("RIGHT");
    return "Right movement";
}

std::string mockController::upwardMovement()
{
    receiver.send("UP");
    return "Upward movement";
}

std::string mockController::downwardMovement()
{
    receiver.send("DOWN");
    return "Downward movement";
}
```

Picture 26: In the picture above the second part of mockController.cpp is shown.

In the picture above the implementation of the movement functions are shown. When one of these 6 functions are triggered. When one of the functions gets triggered they send a message to the receiver, who will than send it through the communication line to the drone for it to execute the right commands updating the power distribution of the motors and the coordinates on the gps.

```cpp
void mockController::printControls() const
{
    std::cout << "The controls are:\n"
              << " -w = move forward\n"
              << " -s = move backward\n"
              << " -a = move left\n"
              << " -d = move right\n"
              << " -u = move up\n"
              << " -j = move down\n"
              << " -q = landing the drone and shutting it off\n";
}
```

Picture 27: In the picture above the last part of mockController.cpp is shown

In the picture above the implementation of the printControls function is shown. It is just a bunch of line that will be printed onto the terminal to make clear to the user what you can do to move the drone and which button will shut down the drone. This is also why it is put in private because this is something that shouldn't be edited by any other class.

7. Verification

In this part of the report the test cases are going to be showed to verify the code working.

First the mock-up of the interfaces connected to the drone class will be shown, after that every test will be shown and explained.

```
#ifndef MGPS_H
#define MGPS_H

#include "IGps.h"

#include <gmock/gmock.h>

class MGps : public IGps
{
public:
    MOCK_METHOD3(updatePosition, void(int diffX, int diffY, int diffHeight));
    MOCK_CONST_METHOD0(getX, int());
    MOCK_CONST_METHOD0(getY, int());
    MOCK_CONST_METHOD0(getHeight, int());
    MOCK_METHOD0(landing, void());
};

#endif
```

Picture 28: In the picture above the mockup of the gps interface is shown

In the picture above the IGps class has been reworked into a mock class used for test cases. Every function got a mock method, with the number standing behind the mock showing if the methods use a parameter and if they do it is specified how many parameters they use. So every method uses not a single parameter except for the MOCK_METHOD that is made for the updatePosition function. That method uses 3 parameters, 1 for diffX, 1 for diffY and 1 for diffHeight.

```
#ifndef MMOTOR_H
#define MMOTOR_H

#include "IMotor.h"

#include <gmock/gmock.h>

class MMotor : public IMotor {
public:
    MOCK_METHOD1(setMotorPower, void(int power));
    MOCK_CONST_METHOD0(getMotorPower, int());
};

#endif
```

Picture 29: In the picture above the the mockup of the motor interface is shown

In the picture above the IMotor class has been reworking into a mock class used for test cases. Every function of the interfact got a mock method, with the getMotorPower not needing a parameter, so it stays at 0 and setMotorPower needing 1 parameter for int power.

Now that the mockups have been showed next the setup of the test environment and the different tests will be showed with a small explanation about what they will do. This will be shown on the next page.

```
class DroneTest : public ::testing::Test
{
protected:
    DroneTest() {}
    virtual ~DroneTest() {}

    MGps gps;
    MMotor MotorRB;
    MMotor MotorLB;
    MMotor MotorRF;
    MMotor MotorLF;
};
```

Picture 30: In the picture above the setup of the testing environment is shown

In the picture above the test environment is shown. It makes sure that all of the objects will get created before the tests get executed like the gps and different motors used by the system.

```
TEST_F(DroneTest, StartDrone_startup_test)
{
    EXPECT_CALL(MotorRB, setMotorPower(MOTOR_STARTUP_POWER));
    EXPECT_CALL(MotorLB, setMotorPower(MOTOR_STARTUP_POWER));
    EXPECT_CALL(MotorRF, setMotorPower(MOTOR_STARTUP_POWER));
    EXPECT_CALL(MotorLF, setMotorPower(MOTOR_STARTUP_POWER));

    EXPECT_CALL(MotorRB, setMotorPower(MOTOR_FLOATING_POWER));
    EXPECT_CALL(MotorLB, setMotorPower(MOTOR_FLOATING_POWER));
    EXPECT_CALL(MotorRF, setMotorPower(MOTOR_FLOATING_POWER));
    EXPECT_CALL(MotorLF, setMotorPower(MOTOR_FLOATING_POWER));

    EXPECT_CALL(gps, updatePosition(0, 0, 5));

    StartDrone(gps, MotorRB, MotorLB, MotorRF, MotorLF);
}
```

Picture 31: In the picture above the startup test is shown

In the picture above the startup function of the system is tested. During the execution it checks for the power of the motors and after it is executed it checks for the position of the drone to check if it was executed correctly.

Because it is testing a function the call of the test is called TEST_F() and not TEST(). This will be the same for all the other tests. If they test values during the execution of a function they shall use TEST_F() and if they aren't testing the execution of a function the will be defined with TEST()

```
TEST_F(DroneTest, StopDrone_landing_test)
{
    EXPECT_CALL(MotorRB, setMotorPower(MOTOR_LANDING_POWER));
    EXPECT_CALL(MotorLB, setMotorPower(MOTOR_LANDING_POWER));
    EXPECT_CALL(MotorRF, setMotorPower(MOTOR_LANDING_POWER));
    EXPECT_CALL(MotorLF, setMotorPower(MOTOR_LANDING_POWER));

    EXPECT_CALL(MotorRB, setMotorPower(MOTOR_OFF));
    EXPECT_CALL(MotorLB, setMotorPower(MOTOR_OFF));
    EXPECT_CALL(MotorRF, setMotorPower(MOTOR_OFF));
    EXPECT_CALL(MotorLF, setMotorPower(MOTOR_OFF));

    EXPECT_CALL(gps, landing());

    StopDrone(gps, MotorRB, MotorLB, MotorRF, MotorLF);
}
```

Picture 32: In the picture above the shutting down test is shown

In the picture above the stop drone function of the system is tested. During the execution it checks for the power of the motors and after it is executed it checks for the position of the drone to check if it was executed correctly.

```
TEST(DroneTest, Initial_Value_Test)
{
    fakeGps g;
    EXPECT_EQ(0, g.getX());
    EXPECT_EQ(0, g.getY());
    EXPECT_EQ(0, g.getHeight());
}
```
Picture 33: In the picture above a test is executed to check the coordinates

In the picture above a test is executed that tests if the coordinates of the gps get printed correctly if they don't get any value into them. They should just print 0 and nothing else because the drone shouldn't have moved.

```
TEST(DroneTest, Update_Position_Applies_Changes_Test)
{
    fakeGps g;
    g.updatePosition(64, 98, 12);
    EXPECT_EQ(64, g.getX());
    EXPECT_EQ(98, g.getY());
    EXPECT_EQ(12, g.getHeight());
}
```
Picture 34: In the picture above a test is executed to check the coordinates

In the picture above a test is executed that tests if the coordinates of the gps get printed correctly if the coordinates get adjusted. The position gets updated to some coordinates chosen by the tester and then when the test gets executed it checks if those coordinates are also correctly put into the system.

```
TEST(DroneTest, Enforce_Min_Height_Test)
{
    fakeGps g;
    g.updatePosition(0, 0, -5);
    EXPECT_EQ(5, g.getHeight());
}
```
Picture 35: In the picture above a test is executed to check the coordinates

In the picture above a test is executed that tests if the coordinates of the gps get printed correctly with the height in mind. The system has a minimum flying height of 5, and if that would be broken it should automatically go to a height of 5. That gets tested with this test.

```
TEST(DroneTest, Landing_Coordinates_Test)
{
    fakeGps g;
    g.updatePosition(12, 12, 10);
    g.landing();

    EXPECT_EQ(12, g.getX());
    EXPECT_EQ(12, g.getY());
    EXPECT_EQ(0, g.getHeight());
}
```
Picture 36: In the picture above a test is executed combining reading the coordinates and shutting off the system

In the picture above a test is executed to check if the coordinates update correctly when the system gets shut down. This means that the system manually gets some coordinates and after that the landing sequence gets executed. After that is done the coordinates are checked, because they should give the same coordinates that were given at the start of the test, except for the height, that should be at 0.